LinkUp — Comprehensive Backend Architecture & Engineering Blueprint

1-on-1 Real-Time Audio/Video Calling, Per-Minute Coin Billing, Digital Wallet Ledger, and Messaging Engine

Target Mobile Client: Flutter (Android/iOS)	**Target Environment:** Production / Cloud Native
Author / Architecture: Principal Engineering Team	**Status:** Approved Architecture Specification
Primary Core Domain: Real-Time RTC & Micro-Transactions	**Date:** October 2026

1. Executive Summary & Platform Domain

LinkUp is an interactive mobile platform connecting users with creators for real-time engagement. The system incorporates high-frequency monetization where creators charge variable rates (e.g., 200–500 coins/min) for audio/video interactions, private photo unlocks, and direct chat. The backend is responsible for absolute authority over session authenticity, coin deduction, wallet security, and WebRTC signaling.

Key Platform Pillars:
- **Real-Time RTC Engine:** 1-on-1 audio/video sessions using secure ephemeral tokens with sub-200ms latency.
- **Live Per-Minute Billing Worker:** Server-enforced coin deduction without reliance on client timekeeping.
- **Financial Grade Wallet Ledger:** Double-entry accounting for all coin purchases, holds, deductions, and creator payouts.
- **Admin Kill-Switches:** Real-time controls to toggle balance freezing, disable video calling, and manage UPI endpoints.

2. Recommended Technology Stack & Infrastructure

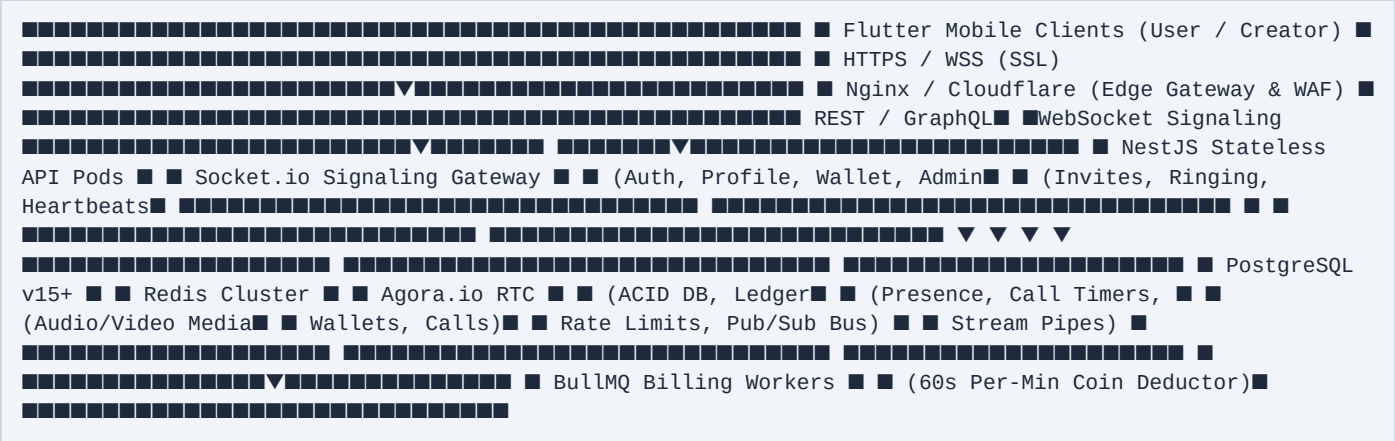
The stack has been selected to guarantee ACID data integrity, high-throughput WebSocket concurrency, and minimal operational overhead.

Layer	Technology	Strategic Rationale
Backend API & Signaling	**Node.js (NestJS / TypeScript)** *Alternative: Go (Golang)*	Enterprise modular DI architecture, native WebSocket gateway handling, shared DTO typings with mobile client.
Primary Database	**PostgreSQL (v15+)** with Prisma or Drizzle ORM	Strict ACID compliance, row-level locking, foreign-key integrity essential for financial ledgers and call records.
In-Memory State & Cache	**Redis (v7+)**	Stores creator online/busy presence, active call heartbeats, rate limiting, and pub/sub message distribution.
Real-Time Media Engine	**Agora.io RTC SDK** *Alternative: LiveKit Cloud*	Industry standard for low-latency 1-to-1 mobile calling with dynamic cryptographic token auth and server-side kick API.

WebSockets Gateway	**Socket.io + Redis Adapter**	Bi-directional signaling for call invites, ringing states, minute-by-minute balance updates, and instant chat.
Payment Gateway	**Razorpay / Cashfree / PhonePe**	Supports Indian UPI Intent, Dynamic QR, Net Banking, and webhooks with cryptographically signed payload verification.
Object Storage & CDN	**Cloudflare R2 / AWS S3** + Cloudflare CDN	Zero-egress fee media storage for user gallery, avatars, and private locked photos served via signed temporary URLs.
Task Queue & Crons	**BullMQ (Redis-backed)**	Handles asynchronous webhook reconciliation, push notifications, and daily creator payout batch consolidation.

3. High-Level Architectural Flow

The architecture separates stateless HTTP request processing from stateful WebSocket signaling connections. Stateful connections communicate across horizontally scaled API nodes using Redis Pub/Sub.



4. Relational Database Schema Design (PostgreSQL)

All tables utilize UUID primary keys generated by the server. Strict foreign keys and indexing are applied on high-traffic query paths.

Table: users

Column	Type	Constraints	Description
id	UUID	PK, gen_random_uuid()	Primary unique identifier
phone	VARCHAR(15)	UNIQUE, Indexed	Primary authentication phone number
email	VARCHAR(255)	UNIQUE, Nullable	Alternative email login
role	ENUM	'USER','CREATOR','ADMIN'	Role-based access permissions
is_verified	BOOLEAN	DEFAULT false	Identity and phone verification status
is_active	BOOLEAN	DEFAULT true	Active / banned / suspended account flag
created_at	TIMESTAMPTZ	DEFAULT now()	Registration timestamp

Table: creators

Column	Type	Constraints	Description
id	UUID	PK	Unique Creator identifier
user_id	UUID	FK -> users.id, UNIQUE	Links to base user entity
tier	ENUM	'SILVER','GOLD','VIP'	VIP badge tier displayed in app
is_online	BOOLEAN	DEFAULT false, Indexed	Live online presence flag
is_available	BOOLEAN	DEFAULT true, Indexed	False if creator is currently on another call
video_call_price	INT	DEFAULT 500, >= 0	Coins deducted per minute for video calling
voice_call_price	INT	DEFAULT 300, >= 0	Coins deducted per minute for voice calling
photo_price	INT	DEFAULT 200, >= 0	Coins required to unlock locked photo message
rating	NUMERIC(3,2)	DEFAULT 5.00	Average rating (e.g. 4.9)
review_count	INT	DEFAULT 0	Total review count (e.g. 128)

Table: wallets

Column	Type	Constraints	Description
id	UUID	PK	Wallet unique identifier
user_id	UUID	FK -> users.id, UNIQUE	Owner user entity
balance	INT	DEFAULT 0, CHECK(>= 0)	Available coins balance (strictly non-negative)
reserved_balance	INT	DEFAULT 0	Coins held during an active ongoing call
version	INT	DEFAULT 0	Optimistic concurrency locking version
is_frozen	BOOLEAN	DEFAULT false	Emergency freeze flag (set by admin)

Table: wallet_transactions (Immutable Ledger)

Column	Type	Constraints	Description
id	UUID	PK	Transaction identifier
wallet_id	UUID	FK -> wallets.id, Indexed	Target wallet

type	ENUM	'PURCHASE','CALL_USAGE','PHOTO_PURCHASE','REFUND','SESSION_HOLD'	Transaction category
coins	INT	NOT NULL (Signed)	+Credit for recharges, -Debit for usage
amount_inr	NUMERIC(10,2)	Nullable	Actual fiat amount for recharges (■199 - ■699)
balance_after	INT	NOT NULL	Snapshot of wallet balance after transaction
status	ENUM	'COMPLETED','PENDING','REVERTED'	Execution status of ledger record
reference_id	VARCHAR(100)	Nullable, Indexed	Foreign CallSessionId, OrderId, or MediaId

Table: call_sessions

Column	Type	Constraints	Description
id	UUID	PK	Call session identifier
caller_id	UUID	FK -> users.id, Indexed	Initiator (User)
creator_id	UUID	FK -> creators.id, Indexed	Recipient (Creator)
call_type	ENUM	'VOICE','VIDEO'	Call modality
rate_per_min	INT	NOT NULL	Rate frozen at initiation (e.g. 500 coins/min)
status	ENUM	'INITIATED','RINGING','CONNECTED','COMPLETED','MISSED','REJECTED'	Call lifecycle state
duration_secs	INT	DEFAULT 0	Total billable connected duration
total_coins	INT	DEFAULT 0	Total coins debited from caller
agora_channel	VARCHAR(100)	UNIQUE	Generated Agora channel token scope
end_reason	VARCHAR(50)	Nullable	'USER_HANGUP','INSUFFICIENT_BALANCE', etc.

5. Critical Engineering Flow: Real-Time Call Billing Engine

In a pay-per-minute platform, the client device cannot be trusted to report call duration or deduct currency. The backend server maintains absolute authority over session validity, rate enforcement, and channel termination.

Step 1: Pre-Call Feasibility Check

When caller presses 'Call', the server verifies: (a) Creator is online and available, (b) Caller balance is >= 1 minute rate (e.g., 500 coins), (c) Platform video killswitch is active. If verified, the server puts creator in BUSY state and generates temporary Agora tokens.

Step 2: Session Reservation (Hold)

To prevent double-spending across multiple tabs or simultaneous requests, the server places an atomic hold of 1 minute's worth of coins into `wallets.reserved_balance`.

Step 3: Connect Event & Heartbeat Timer

Once the creator answers, the Agora channel reports active media exchange. The server initializes a Redis recurring timer (`call_timer:{callId}`) with a 60-second tick interval.

Step 4: Minute-by-Minute Atomic Debit

Every 60 seconds, a dedicated BullMQ worker fires. It executes an atomic SQL transaction: debiting caller wallet, crediting creator wallet (minus platform commission), and emitting `wallet:balance_update` via WebSockets to the caller app.

Step 5: Low-Balance Warning & Grace Period

If caller's remaining balance is insufficient for the upcoming minute, the server fires `call:balance_warning` (30 seconds remaining). An in-call recharge bottom-sheet is prompted on the user's screen.

Step 6: Automated Channel Termination

If zero balance is reached and no recharge occurs before the 60s window lapses, the server directly invokes Agora REST API (`POST /v1/projects/{appId}/kicking-rule`) to sever the audio/video media pipe, cancels the Redis timer, resets creator state to AVAILABLE, and saves final call records.

CRITICAL: Optimistic Concurrency Locking against Double-Spend
Under high traffic, a user might open multiple sockets. To guarantee coin balances never drop below zero, all coin deduction queries must execute with strict atomic predicates:
UPDATE wallets SET balance = balance - :rate, version = version + 1 WHERE user_id = :userId AND version = :currentVersion AND balance >= :rate;
If updated rows == 0, the transaction aborts and the call is immediately terminated.

6. WebSocket Signaling & Event Specification

All WebSocket connections require standard Bearer JWT authentication upon handshake. Payload exchanges occur over structured JSON.

Event Name	Direction	Payload Content	Operational Purpose
`presence:heartbeat`	Client -> Svr	`{ status: 'online' }`	Sent every 30s; keeps user/creator presence fresh in Redis.
`call:invite`	User -> Svr	`{ creatorId, callType }`	User initiates call; server checks balance & triggers ringing.

`call:incoming`	Svr -> Creator	`{ callId, callerName, photo }`	Displays incoming ringing screen on Creator app.
`call:accept`	Creator -> Svr	`{ callId }`	Creator answers; generates Agora channel tokens.
`call:connected`	Svr -> Both	`{ callId, agoraToken, channel }`	Both parties join Agora channel; starts billing timer.
`call:balance_warning`	Svr -> User	`{ secondsRemaining: 30 }`	Warns caller that coins are about to expire.
`call:ended`	Svr -> Both	`{ callId, duration, totalCoins }`	Terminates UI and displays end summary dialogue.
`chat:message`	Bi-directional	`{ id, convId, content, isLocked }`	Transmits real-time text or pay-to-unlock media.

7. RESTful API Blueprint Specification

Standard JSON over HTTPS. Status codes: 200/201 Success, 400 Validation Error, 401 Unauthorized, 402 Insufficient Balance, 403 Forbidden.

Endpoint	Method	Auth	Functionality & Parameters
`/api/v1/auth/phone/send-otp`	**POST**	Public	Generates and dispatches 6-digit SMS OTP via gateway.
`/api/v1/auth/phone/verify-otp`	**POST**	Public	Verifies OTP. Returns { accessToken, refreshToken, isNewUser }.
`/api/v1/auth/email/login`	**POST**	Public	Email & password authentication for fallback login.
`/api/v1/profile/me`	**GET**	User	Fetches profile details, interests, 6-photo gallery, and balance.
`/api/v1/profile/onboarding`	**PUT**	User	Saves progressive state across multi-step profile onboarding.
`/api/v1/profile/gallery/upload`	**POST**	User	Generates S3/R2 presigned upload URL for direct client media push.
`/api/v1/creators`	**GET**	User	Paginated creator feed. Filters by tier (VIP/Gold), tags, languages.
`/api/v1/creators/:id`	**GET**	User	Detailed profile, video/voice pricing, rating, response time.
`/api/v1/creators/:id/like`	**POST**	User	Toggles like/saved status for creator discovery drawer.
`/api/v1/wallet/balance`	**GET**	User	Returns current coin balance, freeze state, and reserved balance.
`/api/v1/wallet/packages`	**GET**	User	Lists official 6 coin packages (200 to 700 coins / ■199 to ■699).
`/api/v1/wallet/order/create`	**POST**	User	Creates Razorpay/Cashfree order token for selected coin package.
`/api/v1/wallet/order/verify`	**POST**	User	Validates payment signature, writes ledger entry, credits coins.
`/api/v1/wallet/transactions`	**GET**	User	Paginated transaction history (Purchases, Call debits, Refunds).
`/api/v1/admin/settings`	**GET / PUT**	Admin	Controls isVideoConnectionEnabled, isFreezeBalance, and UPI ID.

8. Security, Content Protection & Anti-Fraud Rules

• **Short-Lived Ephemeral RTC Tokens:** Agora tokens must never be hardcoded on mobile clients. Tokens are minted by the backend dynamically with a 24-hour max lifespan tied strictly to a specific channel name and user integer UID.

• **Protected Media Storage:** Creator private/locked photos are stored in private Cloudflare R2 buckets with zero public access. The backend issues temporary HMAC-signed URLs (valid for 60 seconds) only after verifying coin deduction.

• **Client-Side Screenshot & Screen Recording Guard:** Android clients must set FLAG_SECURE on active call screens to protect creator privacy against unauthorized captures.

• **Single Active Call Enforcement:** Redis maintains an active call session lock per user and creator. If a user attempts to dial while already registered in a call session, the API rejects the request immediately.

• **Idempotent Webhooks:** Payment gateway webhooks require strict cryptographic signature verification (`HMAC-SHA256`) and idempotency checks using the payment transaction ID to prevent duplicate coin allocation on retries.

9. Infrastructure & 4-Week Implementation Roadmap

Hosting Strategy:

• **Phase 1 (MVP / Beta Launch):** Containerized backend deployed on **DigitalOcean / AWS Lightsail** ($20–$40/mo). Managed database via **Supabase PostgreSQL** with serverless **Upstash Redis**. Media hosted on **Cloudflare R2**.

• **Phase 2 (Production Scale):** Auto-scaling **AWS ECS Fargate** cluster behind Application Load Balancer. **AWS Aurora PostgreSQL (Multi-AZ)** with **ElastiCache Redis Cluster**.

Phase	Timeline	Core Deliverables & Milestones
Milestone 1	Week 1	Setup NestJS project, PostgreSQL schema migrations, Phone OTP authentication, and multi-step onboarding profile APIs.
Milestone 2	Week 2	Implement Wallet and immutable Transaction Ledger. Integrate Razorpay UPI gateway and webhook validation. Test race-condition safety.
Milestone 3	Week 3	Build Socket.io signaling server, integrate Agora Server SDK for dynamic tokens, and deploy the 60-second BullMQ call billing worker.
Milestone 4	Week 4	Implement 1-on-1 chat, locked photo unlocking mechanism, Admin controls (`isVideoConnectionEnabled`, `isFreezeBalance`), and final E2E testing.

Engineering Sign-off: This document serves as the formal specification for LinkUp Backend Engineering. All API signatures, database schemas, and billing algorithms conform to the Flutter client contract requirements.